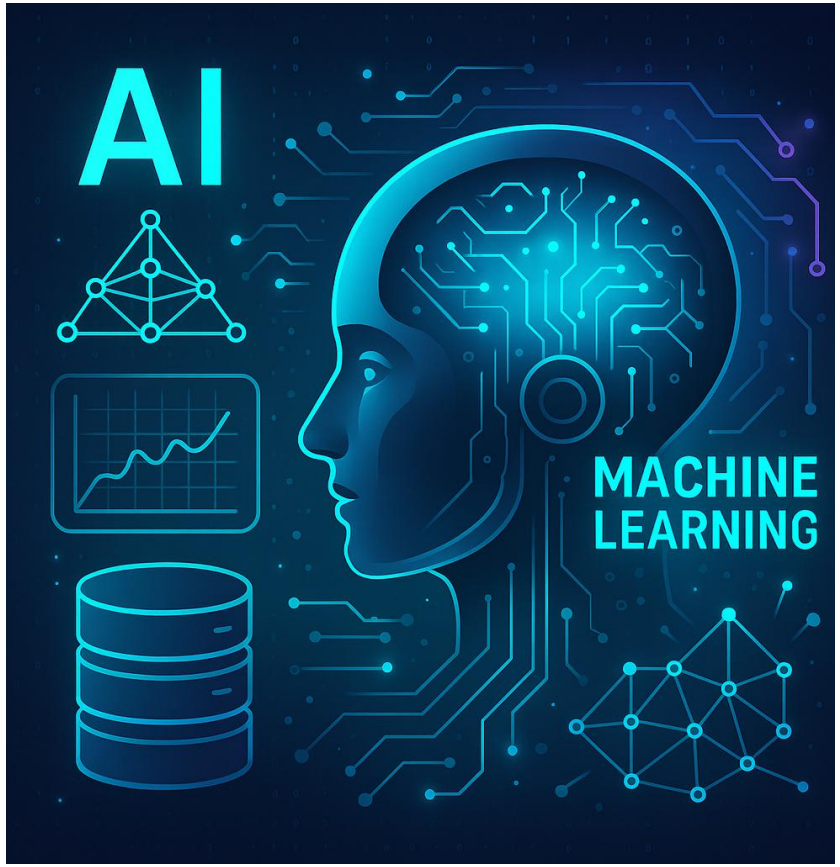
Azure Machine Learning Series

- Overview
- **Data & ETL**
- Compute & Environments
- Modeling & Experimentation
- Deploying & Operating Models
- Team Environment & Enterprise Governance



Azure Machine Learning

Data & ETL

Agenda

- Azure ML Overview Recap
- Datastores and data assets
- Creating datasets
- Data ingestion in Azure ML
- What is ETL in ML
- Transforming data
- Pipelines and automation
- Best practices

Recap
Azure ML Overview

GitHub
https://github.com/danielecolon/Azure-ML/tree/main/1-Overview

YouTube: Azure Machine Learning Step 1 – Overview
https://www.youtube.com/watch?v=Xx9EuqwUoRQ&t=1983s

Demo

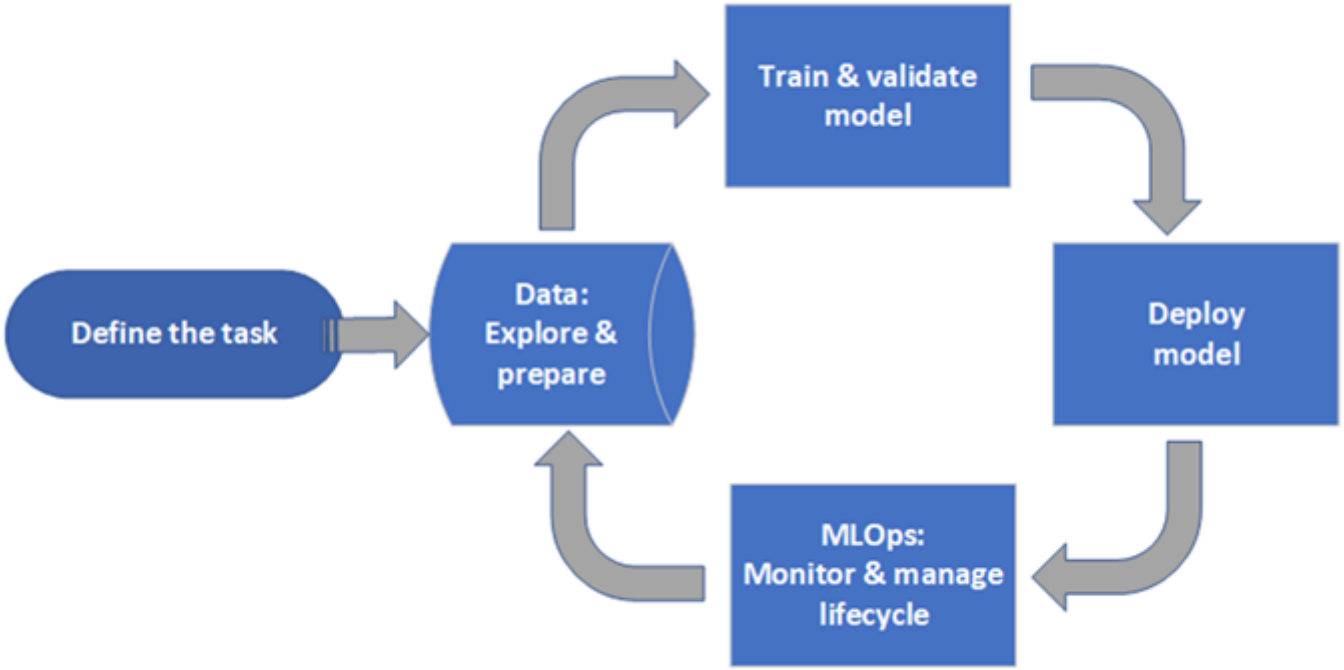
Recap: Azure ML Overview

Creating an Azure ML Studio Workspace

ML Lifecycle

Stages:

- Prepare Data
- Train & Validate
- Deploy
- Monitor



https://learn.microsoft.com/en-us/azure/machine-learning/overview-what-is-azure-machine-learning?view=azureml-api-2

Data concepts in Azure ML

- Datastore
  - Default datastores
- Data types
- URI
- Data asset

https://learn.microsoft.com/en-us/azure/machine-learning/concept-data?view=azureml-api-2

Datastore

Reference to an existing Azure storage account

Supported storage service	Credential-based authentication	Identity-based authentication
Azure Blob Container	✓	✓
Azure File Share	✓	
Azure Data Lake Gen1	✓	✓
Azure Data Lake Gen2	✓	✓

Default datastores

Each workspace contains these default datastores:

Datastore name	Data storage type	Data storage name	Description
workspaceblobstore	Blob container	azureml-blobstore-{workspace-id}	Stores **data uploads**, job code snapshots, and pipeline data cache.
workspaceworkingdirectory	File share	code-{GUID}	Stores data for **notebooks**, compute instances, and prompt flow.
workspacefilestore	File share	azureml-filestore-{workspace-id}	Alternative container for **data upload**.
workspaceartifactstore	Blob container	azureml	Metrics, models, components, **computer & experiment logs**

URI (Uniform Resource Identifier)

Storage location/URI example
Azure Machine Learning Datastore azureml://datastores/<data_store_name>/paths/<folder1>/<folder2>/<folder3>/<file>.parquet
Local computer ./home/username/data/my_data
Public http(s) server https://raw.githubusercontent.com/pandas-dev/pandas/main/doc/data/titanic.csv
Blob storage wasbs://<containername>@<accountname>.blob.core.windows.net/<folder>/
Azure Data Lake (gen2) abfss://<file_system>@<account_name>.dfs.core.windows.net/<folder>/<file>.csv
Azure Data Lake (gen1) adl://<accountname>.azuredatalakestore.net/<folder1>/<folder2>

Demo

Datastore

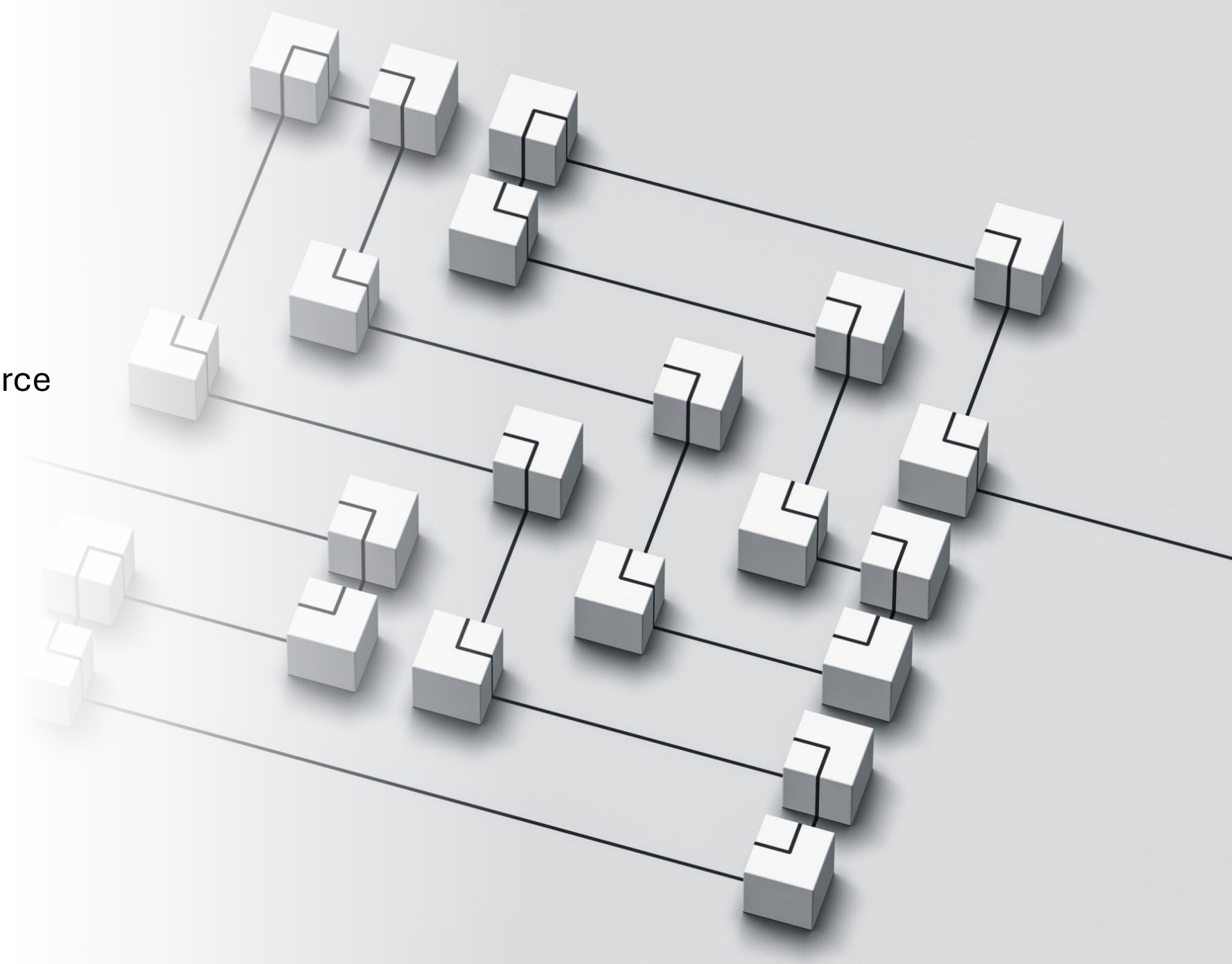
Creating datastore via UI, CLI, and SDK

Data Assets

Reference to the data source location

Data asset benefits:

- Versioning
- Reusable
- Sharable
- Ease-of-use



What is ETL

ETL =

Extract

Transform

Load

ETL in Azure ML

ETL integrates with:

- storage
- compute
- pipelines

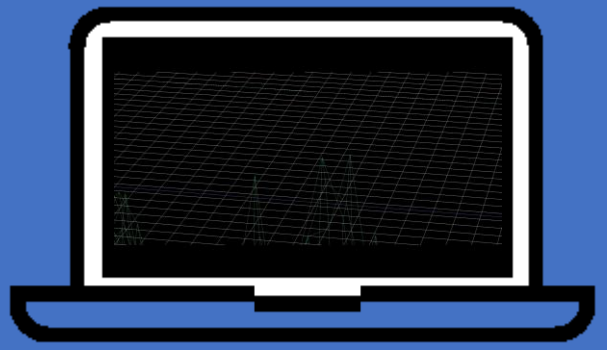
Data types

A URI (storage location) references a file, a folder, or a data table.
ML job input and output definition requires one of these data types:

Type	V2 API	V1 API*
File	uri_file	FileDataset
Folder	uri_folder	FileDataset
Table	mltable	TablularDataset

* V1 API is being deprecated June 30, 2026

Demo



Datasets

Creating datasets via UI, CLI, and SDK

Data types

A URI (storage location) references a file, a folder, or a data table. ML job input and output definition requires one of these data types:

Type	V2 API	V1 API*
File	uri_file	FileDataset
Folder	uri_folder	FileDataset
Table	mltable	TablularDataset

* V1 API is being deprecated June 30, 2026

MLTable Format

MLTable defines:

- schema
- transformations
- data location

YAML

```yaml
$schema: https://azuremlschemas.azureedge.net/latest/MLTable.schema.json
type: mltable

# Supported paths include:
# local: ./<path>
# blob: wasbs://<container_name>@<account_name>.blob.core.windows.net/<path>
# Public http(s) server: https://<url>
# ADLS gen2: abfss://<file_system>@<account_name>.dfs.core.windows.net/<path>/
# Datastore: azureml://subscriptions/<subid>/resourcegroups/<rg>/workspaces/<ws>/datastores/<datastore_name>/

paths:
  - file: abfss://<file_system>@<account_name>.dfs.core.windows.net/<path>/ # a specific file on ADLS
  # additional options
  # - folder: ./<folder> a specific folder
  # - pattern: ./*.csv # glob all the csv files in a folder

transformations:
    - read_delimited:
        encoding: ascii
        header: all_files_same_headers
        delimiter: ","
        include_path_column: true
        empty_as_string: false
    - keep_columns: [col1, col2, col3, col4, col5, col6, col7]
    # or you can drop_columns...
    # - drop_columns: [col1, col2, col3, col4, col5, col6, col7]
    - convert_column_types:
        - columns: col1
          column_type: int
        - columns: col2
          column_type:
            datetime:
                formats:
                    - "%d/%m/%Y"
        - columns: [col1, col2, col3]
          column_type:
            boolean:
                mismatch_as: error
                true_values: ["yes", "true", "1"]
                false_values: ["no", "false", "0"]
      - filter: col('col1') > 32 and col('col7') == 'a_string'
      # create a column called timestamp with the values extracted from the folder information
      - extract_columns_from_partition_format: {timestamp:yyyy/MM/dd}
      - skip: 10
      - take_random_sample:
          probability: 0.50
          seed: 1394
      # or you can take the first n records
      # - take: 200
```

https://learn.microsoft.com/en-us/azure/machine-learning/reference-yaml-mltable?view=azureml-api-2

Data Transformation

- Cleaning
- Filtering
- Feature engineering

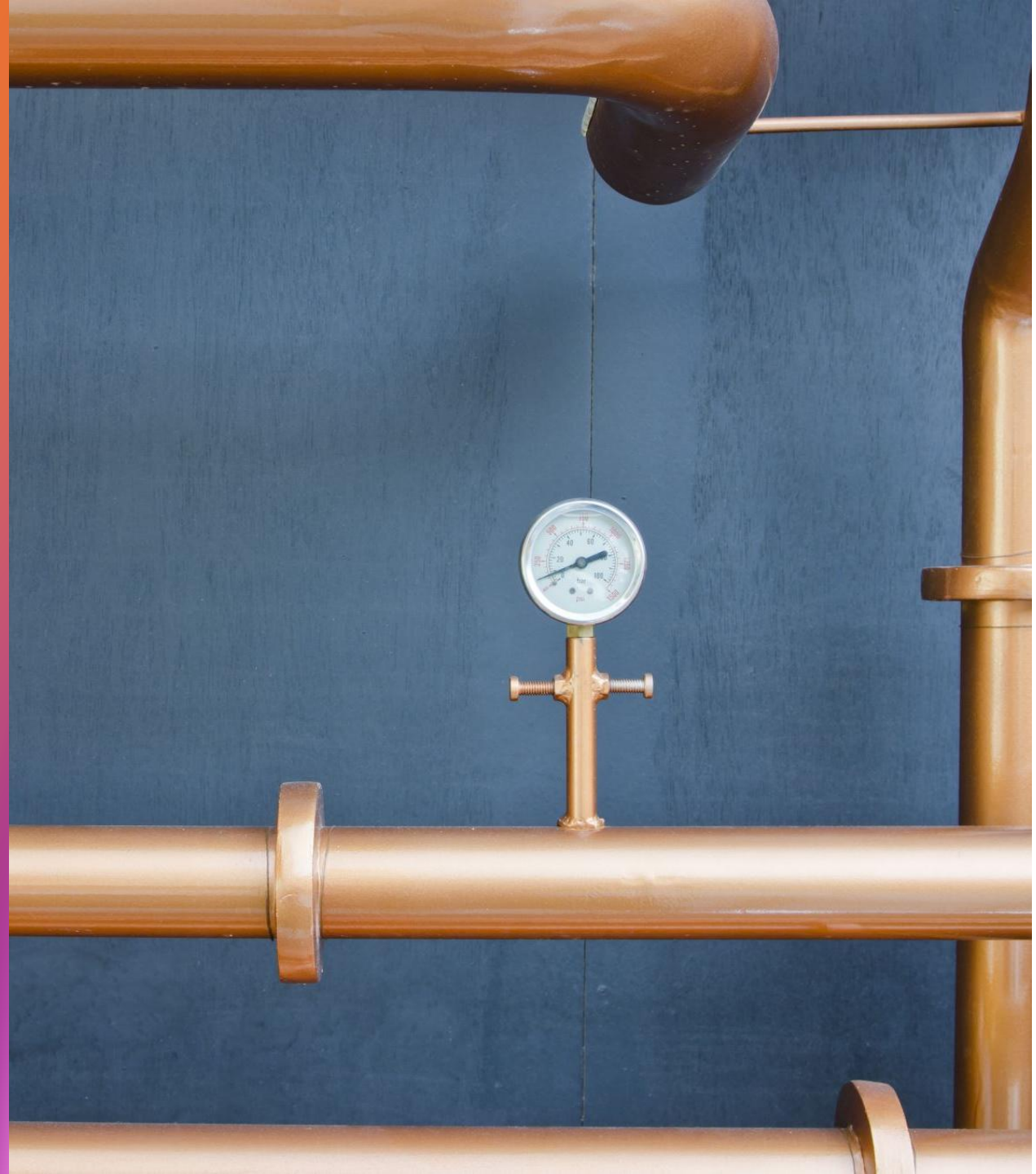
Common Drop Columns Criteria

- High Cardinality Categorical Columns
- High percentage of missing values
- Redundancy/Highly Correlated Columns
- Low Variability (Constant Columns)
- Irrelevant Columns

Using Pandas in Azure ML

Example tasks:

- Filtering rows
- Handling missing values
- Feature engineering



Pipelines

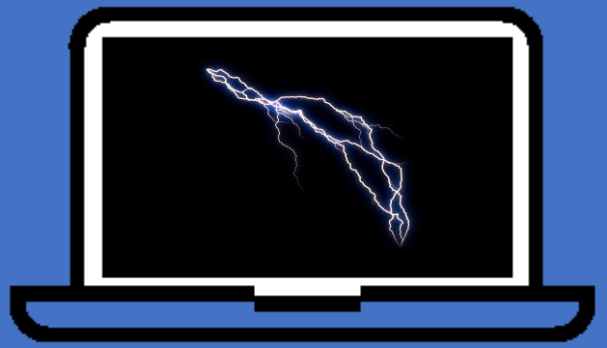
- Automation
- Repeatability
- Modular workflows

Data Versioning

Benefits:

- Reproducibility
- Auditability
- Collaboration

Demo



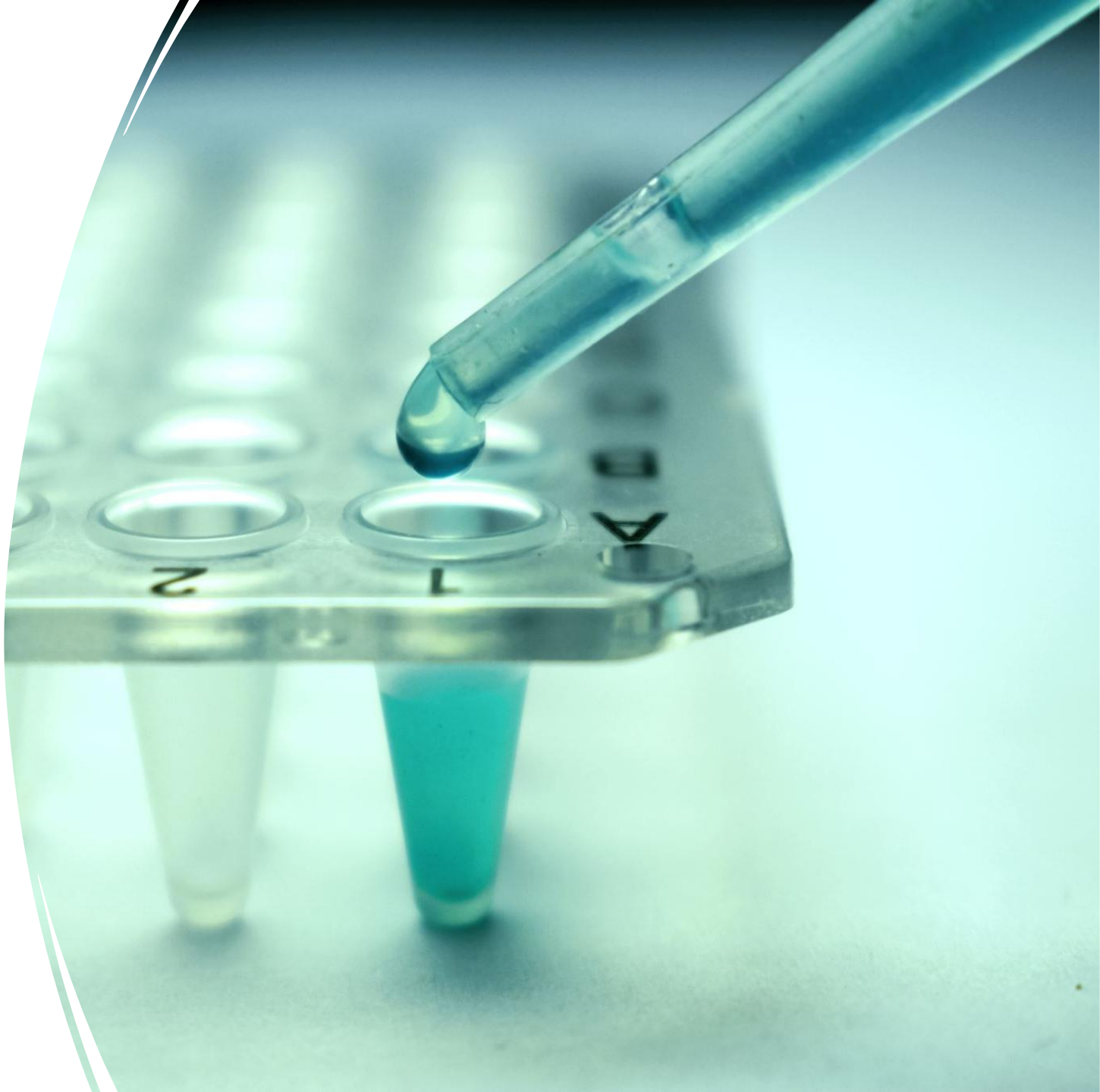
Data Transformations

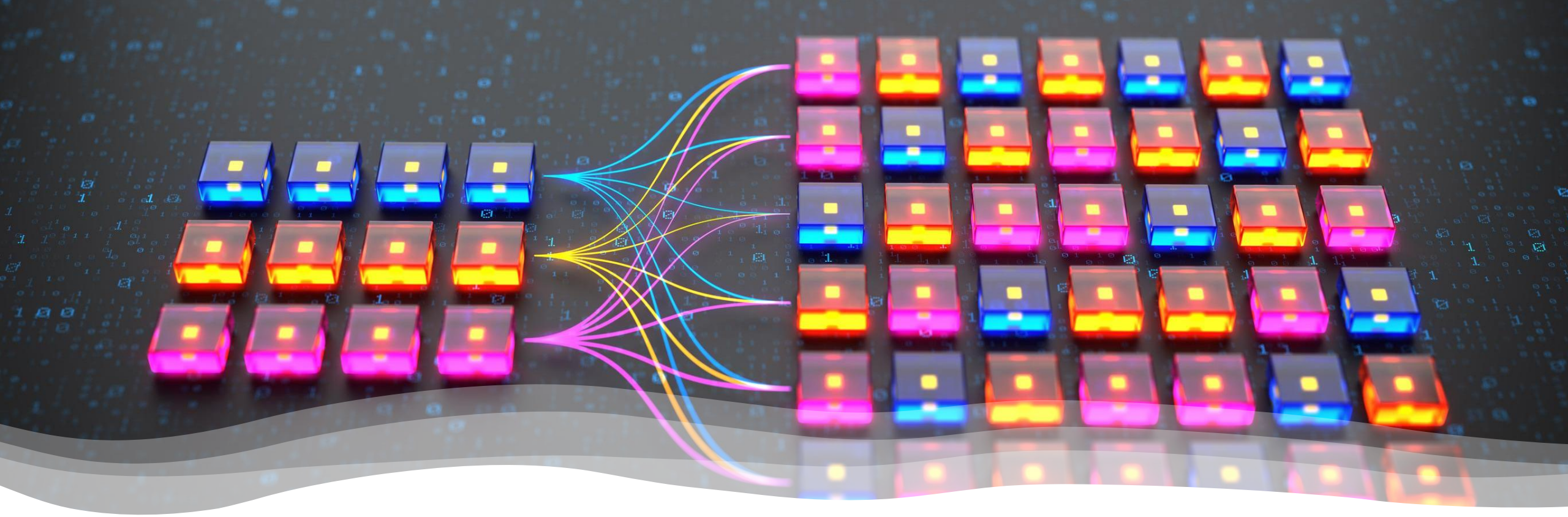
Transformations via Pipelines, Pandas and MLTable

Data Quality

Key factors:

- Completeness
- Consistency
- Accuracy





Scaling ETL

Scaling approaches:

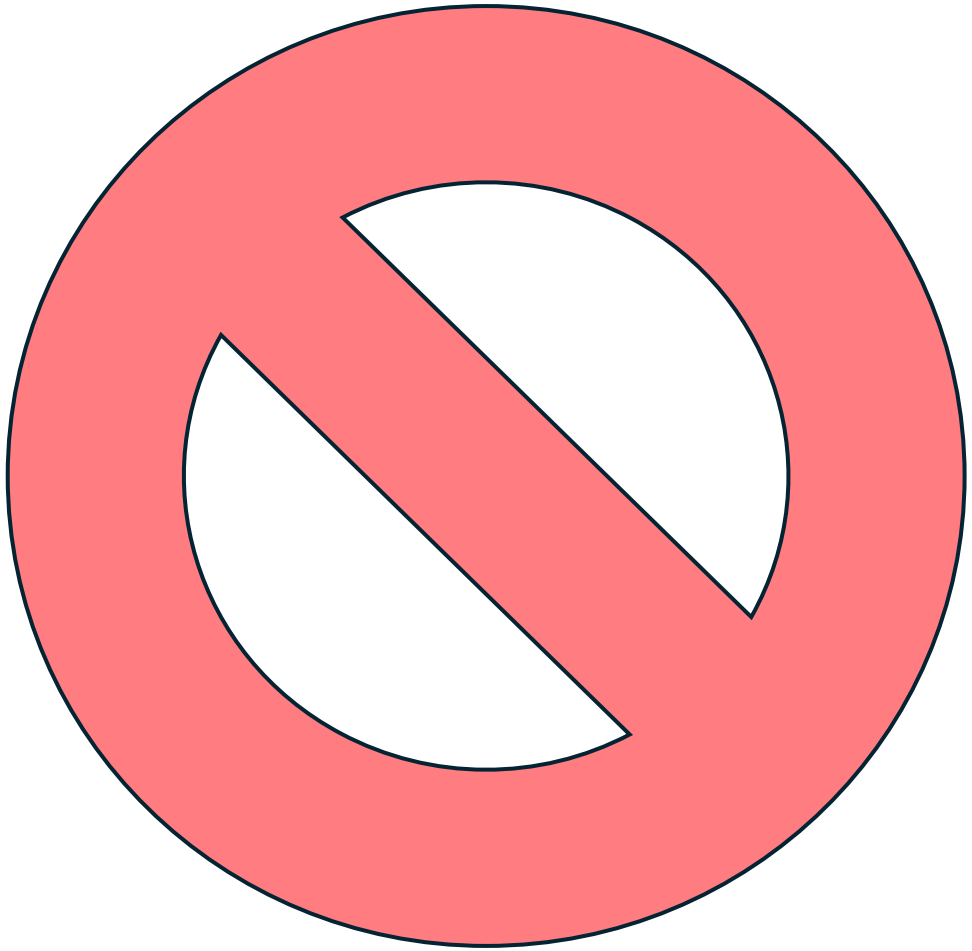
- Parallel processing
- Distributed compute

Best Practices

- Version datasets
- Automate pipelines
- Validate data
- Separate environments

Common Mistakes

- Not versioning data
- Manual data prep
- Ignoring data quality



Storage → ETL → Dataset → Training → Model → Deploy → Monitor

Architecture Flow

Summary

- ETL is critical
- Azure ML simplifies data workflows
- Pipelines enable scalability

References

Daniel Colón LinkedIn

https://www.linkedin.com/in/danielecolon/

Presentation Slides/Code

https://github.com/danielecolon/Azure-ML

Azure Machine Learning documentation

https://learn.microsoft.com/en-us/azure/machine-learning/?view=azureml-api-2